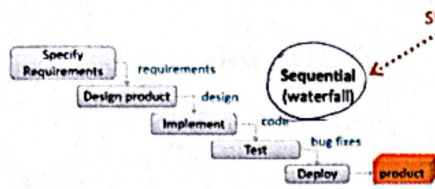


PROJECT MGT

REQUIREMENTS

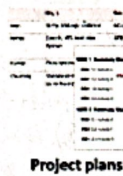


Views software development as a linear process; works well if the project is for a well-understood problem

Planning, scheduling, tracking

Work-breakdown structures

WBS Requirements
WBS 1.1. Develop the R of the system
WBS 1.2. Develop the R of the system
WBS 1.3. Develop the R of the system
WBS 1.4. Develop the R of the system



Project plans



Issue trackers

Includes buffers and milestones



Initial Estimates

Schedule (using inflated estimates)

Task X Task Y

Task X Task Y

Task X Task Y

Task X Task Y

Task X Task Y

Task X Task Y

Task X Task Y

Team structures

egoless team

chief-programmer

strict-hierarchy

大家平等的交流

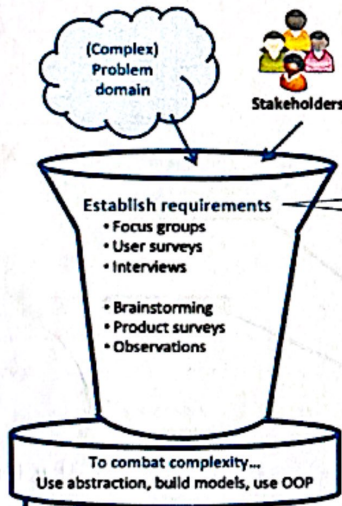
一条为大

分层

Software project

Greenfield (new project, from scratch) iP

Brownfield (build on an existing product) tP



Functional requirements: what the system should do. 应该做的

Non-functional requirements: the constraints under which the system is developed and operated. 应该做的

Desirable qualities: Unambiguous Testable (verifiable) Clear (concise, terse, simple, precise) Correct

Understandable Feasible (realistic, possible) Independent Atomic Necessary Implementation-free

Also, Consistent Non-redundant Complete

to generate many diverse and creative ideas, without evaluating

Specify requirements...

Textual descriptions

Feature list

User stories

Use cases

Prototyping

Conceptual Class Diagrams

Object diagrams

Activity diagrams

...

UI prototypes

Glossary

Supplementary requirements

...

Categorize and prioritize requirements

Requirements Specification

Specifies user's needs

User stories

As a (user type/role) I can (function) so that (benefit)

[Example Use Case]

System: Online Banking System (OBS)

Use case: UC23 - Transfer Money

Actor: User

Preconditions: ...

Guarantees: ...

MSS:

1. User chooses to transfer money.
2. OBS requests for details of the transfer.
3. ...
5. User confirms.
6. OBS transfers the money and displays ...

Extensions:

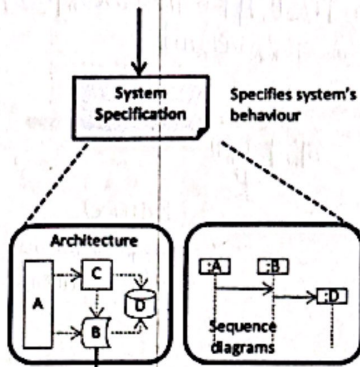
- 3a. OBS detects an error in the entered data.
- 3a1. OBS requests for the correct data.
- 3a2. User enters new data.
- Steps 3a1-3a2 are repeated until ..
- Use case resumes from step 4.

3b. User requests ...

*a. At any time, User chooses to cancel ...

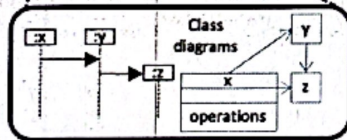
*a1. OBS ...

DESIGN



由上到下
由API到API

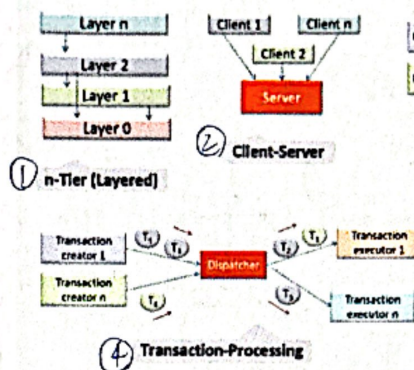
API (Application Programming Interface)



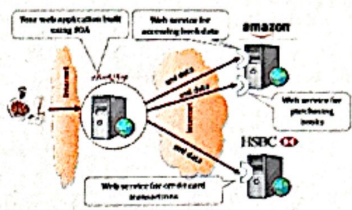
← More agile means less upfront design



Architecture styles:

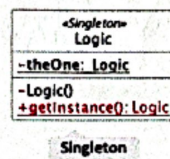


③ Event-Driven



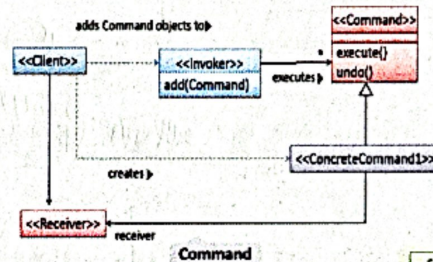
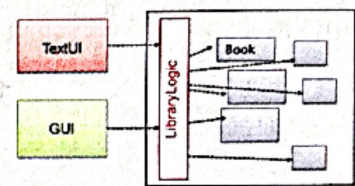
⑤ Service-Oriented

Design Patterns (API?)

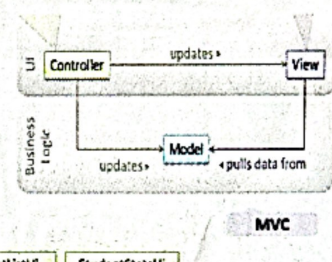


Singleton

Facade



Command



MVC

1. 数据业务逻辑
Model
View (UI显示)
Controller (是MV之间
的Bridge)
→ separation of concerns

Design Principles

- Single responsibility principle (SRP): A class should have one, and only one, reason to change.
- Open-closed principle (OCP): A module should be open for extension but closed for modification.
- Liskov substitution principle (LSP): Derived classes must be substitutable for their base classes.
- Separation of concerns principle (SoC): Separate the code into distinct sections; each address a separate concern.
- Law of Demeter (LoD): An object should only interact with objects that are closely related to it.

A method m of an object O should invoke only the methods of the following kinds of objects:

- ✓ The object O itself
- ✓ Objects passed as parameters of m
- ✓ Objects created/instantiated in m (directly or indirectly)
- ✓ Objects from the direct association of O

Fundamental design qualities:



Abstraction: suppressing lower-level details

- data abstraction
- control abstraction



↓ Coupling: degree of dependence. X is coupled to Y if a change to Y can potentially require a change in X

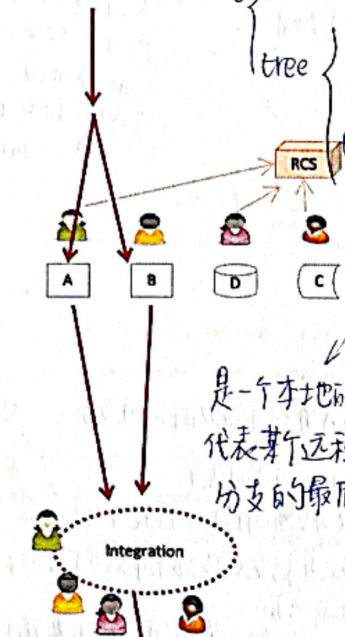
低耦合



↑ Cohesion: how strongly-related and focused the responsibilities of a component are

强相关

IMPLEMENTATION



Revision Control System 版本控制

- RCS**
 - Git is a distributed RCS, not a centralized RCS.
 - Every commit has a unique SHA-1 hash value.
 - Git objects: commits, blobs, trees, tags.
 - Remote-tracking branch ref that represents the state of a corresponding branch in a remote repository.
 - Fork: a copy of a remote repository created on the same hosting service. Different from a clone.
 - pull = fetch + merge
 - Tags: A lightweight tag is just a ref that points directly to a commit; An annotated tag is a full Git object that also stores metadata such as the tagger's name, date, and a message.
 - Interactive rebasing: lets you review and reorganise your recent commits.
 - Merge base: the most recent common ancestor commit of two branches.
 - Squash merge: takes all the changes from the source branch and combines them into a single commit on the destination branch.
 - Fast-forwarding can happen only when the destination branch has not diverged from the source branch.
 - Rebasing moves the base of the source branch to the tip of the destination branch. (重写历史, 变块"迁移")
 - Cherry-picking applies specific commits from one branch onto another.
 - Workflows: forking workflow, feature-branch workflow, centralized workflow.

是一个本地的只读引用, 代表某个远程仓库中对应分支的最后已知状态.

blob = 文件内容 (= 追踪)
tree: 目录结构 (相当于对一个文件夹与块追踪)
Commit: 一次提交, 指向一个 tree
tag: 标记版本号

有 SHA-1, 包含完整的数据
如果要用, 用 -u
把某几个特定的 commit "摘" 出来应用到别的分支

OOP

1. Abstraction: Objects in OOP is an abstraction mechanism because it allows us to abstract away the lower-level details and work with bigger granularity entities.
2. Encapsulation: An object is an encapsulation of some data and related behavior in terms of two aspects.
3. Inheritance: defining a new class based on an existing class. A superclass is said to be more general than the subclass. Multiple Inheritance is when a class inherits directly from multiple classes. (Java 允许多继承)
4. Polymorphism: The ability of different objects to respond, each in its own way, to identical messages.

Overriding vs overloading: Method overriding is when a subclass changes the behavior inherited from the parent class. Method overloading is when there are multiple methods with the same name but different type signatures. (包括: 方法名, 参数列表, 顺序, 数量) 但是参数数量不算!

An interface is a behavior specification.

An abstract class is a class that merely a representation of commonalities among its subclasses. It doesn't make sense to instantiate objects of that class. An abstract method is a method signature without a method implementation.

Substitutability: Every instance of a subclass is an instance of the superclass, but not vice versa. 子类可替代父类!

- Dynamic binding (aka late binding): method calls resolved at runtime.
- Static binding (aka early binding): method calls resolved at compile time.

Code quality guidelines:

- Avoid long methods, deep nesting, complicated expressions, magic literals.
- Make the code 'obvious'. Structure code logically. Do not trip up the reader. Keep it simple. Stupid.
- Avoid premature optimisations.
- Use SLAP (Single Level of Abstraction Principle).
- Make the happy path prominent.
- Follow a standard.
- Naming: Use nouns for things and verbs for actions. Use standard words. Use name to explain. Not too long, not too short. Avoid misleading names.
- Avoid 'unsafe' practices: Use a default branch. Don't recycle variables/parameters. Avoid empty catch blocks. Delete dead code. Minimise scope of variables. Minimise code duplication.
- Comments: Don't state the obvious. Write to the reader. Explain WHAT and WHY, not HOW. (代码本身就已经在告诉你 HOW 了)
- Refactoring: restructuring code in small steps without modifying its external behaviour.

Refactoring is improving structure (not the behavior) of code in small steps.

Examples: Consolidate Conditional Expression, Decompose Conditional, Inline Method, Remove Double Negative, Replace Magic Literal, Replace Nested Conditional with Guard Clauses, Replace Parameter with Explicit Methods, Reverse Conditional, Split Loop, Split Temporary Variable.

Error Handling:

Exception: An event that disrupts the normal flow of the program's execution. Caused by user behavior or issues in the environment. Recover if possible.

Assertions: Assumptions about the program state so that the runtime can verify them. Assertions fail due to programmer errors. Should terminate the execution.

Defensive programming: If you leave room for things to go wrong, they will go wrong. Protect code using assertions, exceptions, logging, and defensive coding.

Documentation:

Go top-down, not bottom-up. Favour for comprehensibility over comprehensiveness. Document minimally but sufficiently.

Integration: 把新开发的模块合在一起

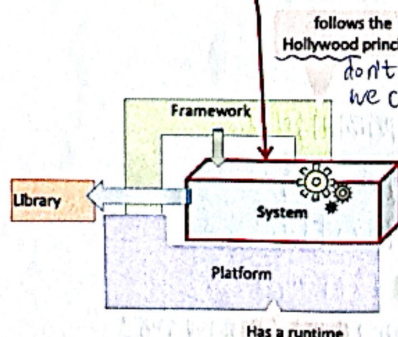
- By frequency: Late and one-time vs early and frequent (preferred).
- By granularity: big-bang vs incremental (preferred).
- By direction: top-down, bottom-up, sandwich (stub, driver).

Continuous Integration (CI) not the same as Continuous Deployment (CD)

Build automation: Gradle is a build automation tool and a dependency management tool.

开发者频繁把代码合并到主分支, 每次合并都触发自动化构建与测试, 确保代码随时都是健康的

在 CI 基础上, 通过测试成功部署到生产环境.



粒度

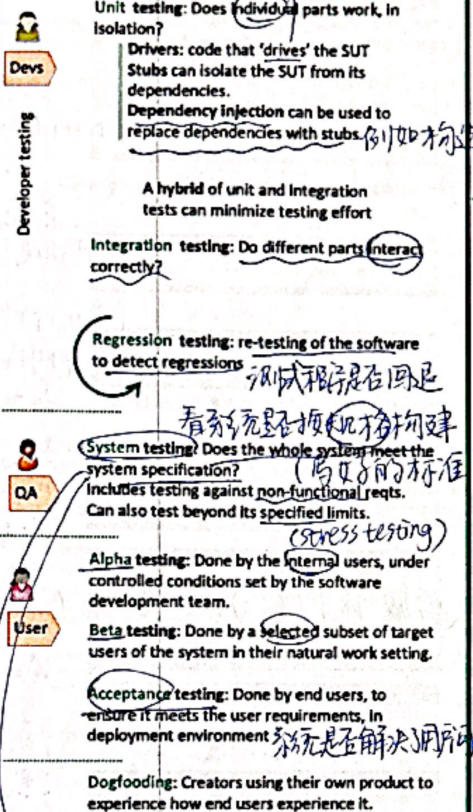
我们能否真正“知道”一个程序是正确的，而不只是相信“它是正确的？”
(伪用的信心 vs. 确定的证明)

CS2103/T - Exam Reference Sheet

大部分 testing =
prove the presence of bugs.
而 formal veri =
prove the absence of bugs.

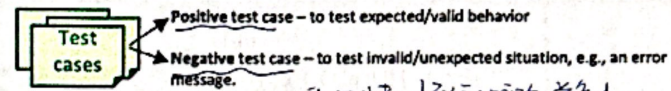
$$QA = \left\{ \begin{array}{l} \text{Validation i.e., are} \\ \text{you building the} \\ \text{right system?} \end{array} \right\} + \left\{ \begin{array}{l} \text{Verification i.e., are} \\ \text{you building the} \\ \text{system right?} \end{array} \right\}$$

模拟上用 Testing



Other V&V techniques:

- **Static analysis:** Static analysis is the analysis of code without executing the code. (Dynamic analysis requires the code to be executed to gather additional info)
- **Formal verification** uses mathematical techniques to prove the correctness of a program.
- **Code reviews**



- Based on knowledge used by test case designer:
- **Black-box** (aka specification-based or responsibility-based) approach: designed exclusively based on the SUT's specified external behavior. [只关心输入与输出]
 - **White-box** (aka glass-box or structured or implementation-based) approach: designed based on implementation, i.e., the code. (完全了解) (开发者自己) $\Rightarrow$ Code coverage
 - **Gray-box** approach: test case design uses some important information about the implementation. (测试者知晓了程序内部实现, 知道系统大致结构与逻辑, 但不深入每一行代码)
- Test case design should aim to be,
- ✓ **Effective** -- find a high percentage of bugs 找bug多少
 - ✓ **Efficient** -- a high rate of bugs found/test cases 你找到的bug中, 有多少是真正的bug.

Heuristics for E&E:

- **Equivalence partitioning:** an EP is a group of test inputs that are likely to be processed by the SUT in the same way. (Avoid testing too many inputs from one partition. Ensure all partitions are tested.)
- **Boundary Value Analysis (BVA)** -- picking test inputs near boundaries (i.e., boundary values) are more likely to find bugs.
- **Combining test inputs:**
 - Strategies: all combinations, all pairs, at least once, random combinations, ...
 - Heuristic: Each Valid Input at Least Once in a Positive Test Case
 - Heuristic: Test Invalid Inputs Individually Before Combining Them

Test coverage

- **Function/method coverage:** based on functions executed e.g., testing executed 90 out of 100 functions. 只问“执行与否”
- **Statement coverage:** based on the number of lines of code executed.
- **Decision/branch coverage:** based on the decision points exercised e.g., an if statement evaluated to both true and false with separate test cases during testing is considered 'covered'.
- **Condition coverage:** based on the boolean sub-expressions, each evaluated to both true and false with different test cases. 确保每个完整的判断条件其结果T/F都被测试到
- **Path coverage** measures coverage in terms of possible paths through a given part of the code executed. $\rightarrow$ 分支的每个结果
- **Entry/exit coverage** measures coverage in terms of possible calls to and exits from the operations in the SUT. 结果 (true / false)

- Approaches:**
- **Scripted testing:** perform testing based on pre-defined set of test cases.
 - **Exploratory testing:** design test cases on-the-fly.
 - **Test-Driven Development (TDD):** evolving functionality and tests in small increments, while writing the test before the functional code.

先清楚你想要什么结果
再去实现它” 根据需求而非代码本身!

SUT: system under test
确保系统中所有函数/方法
被调用到 (entry), 以及所有退出点
被执行到。

Summary

- Function/method = 函数被调用了吗?
 - entry/exit = 每个出入口都走过了吗?
 - Statement = 每行代码都跑过了吗?
 - decision = 每个if的真假都测试过了吗?
 - condition = 每个子条件的真假都测试过了吗?
 - Path = 每条完整执行路径都走过了吗?
- (notation: Control Flow Graph $\Rightarrow$ CFG)

把 decision 拆开, 确保每个独立的子条件, 其T/F都被测试到。

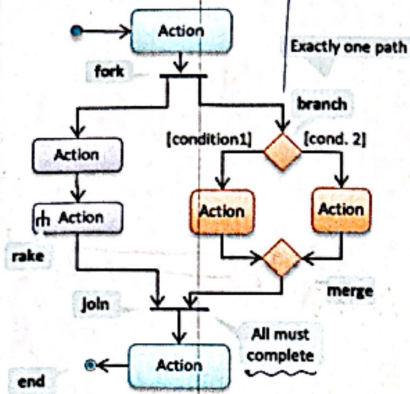
每一行代码

确保条件是互斥的(否则严格使用二分多!)

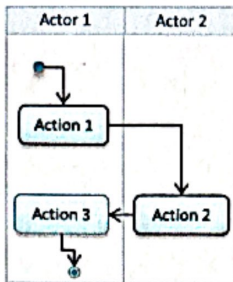
2026/4/29 17:53

CS2103/T - Exam Reference Sheet

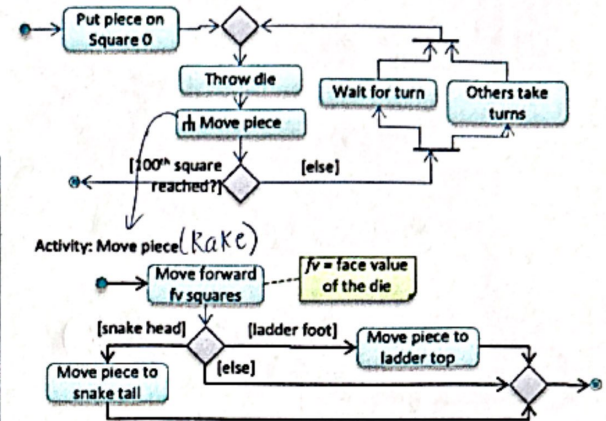
Activity diagrams



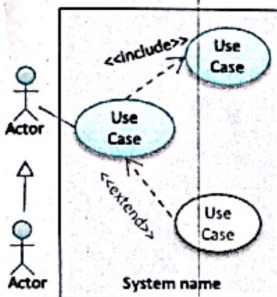
Variant (swimlane diagrams)



[example]

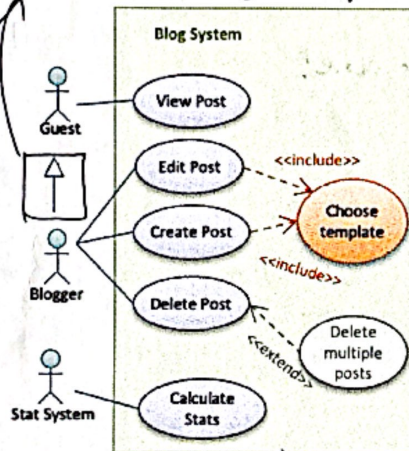


Use case diagrams

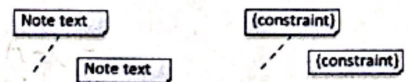


include: underline

Blogger can do anything Guest can do!



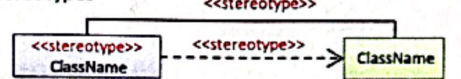
Notes and constraints



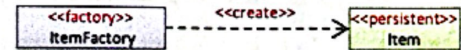
[examples]



Stereotypes



[examples]



注意 include & extend 箭头方向

3 # Amigo = UML

Gang of Four (GoF) = design patterns

Martin Fowler's. UML Distilled -> us

复习 GOF Conventions

java doc: /* * ... *! 大写字母

abcd -> not a good name, but x coding standard.

if(){} ... }

最好不用

(violate coding standard)

assertEquals (only in unit test classes)

* path coverage requires testing every possible execution path, which grows exponentially with the number of conditions

也可以直接 connect 到下一步 (next) 而不用 merge first

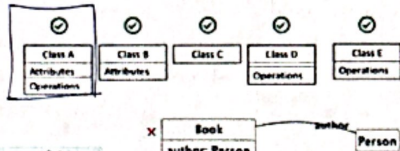
dependency 必须有箭头

* no default multiplicity & visibility

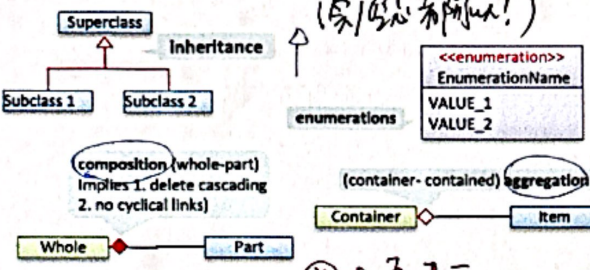
UML

Class diagrams

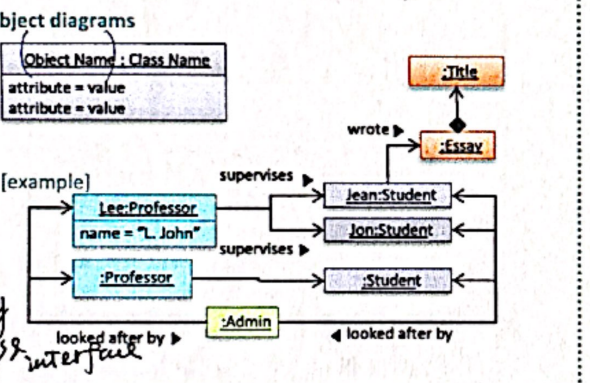
Class name
attributes
methods



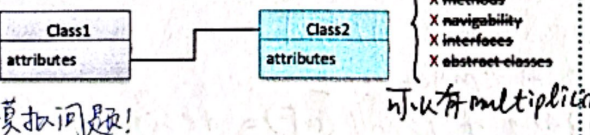
how many of A are associated with 1 of B
multiplicity of A association label role of B
role of A multiplicity of B
association (a structural connection between objects/class)



Inheritance
enumerations
composition (whole-part)
Implies 1. delete cascading
2. no cyclical links
(container-contained) aggregation

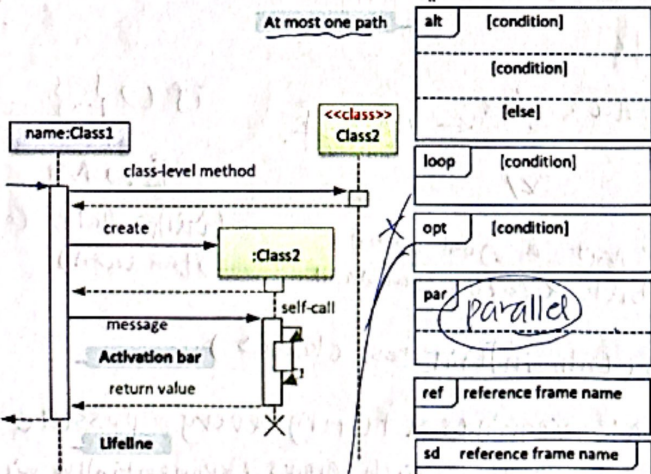


Object diagrams
[example]
Conceptual class diagrams (CCDs) (OODM)



模拟问题!
可以有 multiplicity

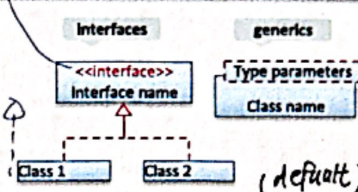
Sequence diagrams



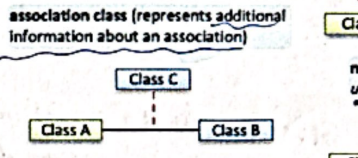
if... else if... else...
At most one path
alt [condition]
[condition]
[else]
loop [condition]
opt [condition]
par parallel
ref reference frame name
sd reference frame name

Car cat = new Cat(); 只有 if, 没有 else
mul 息 息 !!

Class name
visibility name : type multiplicity = default-value
visibility name (parameter-list) : return-type



Interfaces
generics
association class (represents additional information about an association)



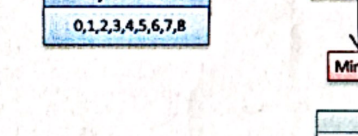
Class A
Class B
Class C

dependency a need for one class to depend on another without having a direct association in the same direction

[examples]



adjacent to
neighbor
played on



Customer
Reservation
LoyaltyProgram

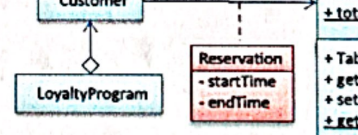
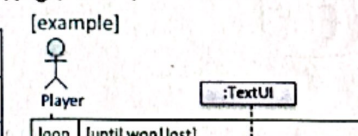
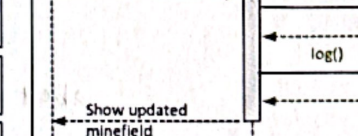


Table
Chair
Minefield
Mine
Cell
MinedCell
MineFreeCell

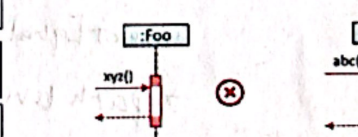
* inheritance & generics



if... else if... else...
[example]



Player
TextUI
MSLogic
Logger



if... else if... else...
At most one path
alt [condition]
[condition]
[else]
loop [condition]
opt [condition]
par parallel
ref reference frame name
sd reference frame name

Car cat = new Cat(); 只有 if, 没有 else
mul 息 息 !!

谁的方法体里写了这行调用语句, 谁就是调用者.